

DIGITAL ASSIGNMENT 1 (DA1)

Course: BCSE306L – B.Tech (Artificial Intelligence)

Faculty: Dr. Vijayaprabakaran K

Academic Year / Semester: 2026-27 / Fall Semester

Project Title:

Adaptive Retrieval-Augmented Generation using Query Complexity and Context Optimization

GitHub Repository:

<https://github.com/aniruddh-batwal07/adaptive-rag-query-aware-retrieval>

(Dataset-loading code has been committed as per DA1 checklist)

Team Members

Member 1: Aniruddh Batwal, 24BRS1122

Member 2: Manisha Bharadwaj, 24BRS1126

1 Problem Identification

1.1 Application Domain

The application domain for this research is Enterprise and Edge Artificial Intelligence, specifically focusing on Retrieval-Augmented Generation (RAG) frameworks for Large Language Models (LLMs), originally introduced by Lewis et al. [1]. While RAG systems aim to mitigate LLM hallucinations by grounding generation in external knowledge bases, conventional RAG pipelines execute a static retrieval strategy (e.g., retrieving a fixed $k = 5$ chunks) regardless of the query’s nature.

1.2 Motivation (Quantified)

As generative AI scales into enterprise production, inference cost and latency have become critical bottlenecks. According to recent industry surveys on LLM deployment, context window processing accounts for the vast majority of token-based API costs. Transmitting thousands of redundant tokens for a simple query (e.g., “What is the capital of France?”) incurs unnecessary financial overhead and adds hundreds of milliseconds to response latency. Conversely, complex multi-hop queries require expansive contexts; however, studies on the “Lost in the Middle” phenomenon [2] demonstrate that feeding excessively large, uncompressed document sets to an LLM actively degrades reasoning accuracy by 20% or more due to attention dilution. Therefore, a dynamic system that gates compute based on query complexity is proposed as an essential framework for sustainable AI deployment.

1.3 Stakeholders and Decisions Supported

- **Enterprise AI Architects:** Will benefit from reduced token utilization and lower infrastructure costs by dynamically skipping expensive retrieval/compression steps for trivial queries.
- **Question Answering (QA) System Users:** Will benefit from sub-200ms latency on simple factual questions and highly accurate, hallucination-free answers on complex analytical questions.
- **Edge AI Developers:** Will be able to deploy complex knowledge assistants on resource-constrained hardware (e.g., 8GB VRAM) because the proposed context compression module aims to prevent context-window overflow.

2 Literature Survey

The following table compares 15 recent and relevant research papers focusing on dynamic retrieval, query routing, and context compression. Click the reference number to view the official PDF.

Ref.	Year	Dataset	Method / Architecture	Key Metric & Value	Stated Limitation
[1]	2024	QA (Single & Multi-hop)	Adaptive-RAG: Classifier-based complexity routing	Latency reduced by ~40%, comparable EM	Router heavily depends on training distribution
[2]	2025	5 QA benchmarks	RouteRAG: RL-based hybrid retrieval routing	Retrieval API calls minimized	RL training is computationally expensive
[3]	2026	Geriatric Health QA	FusionGraphRAG: Intent-aware fuzzy logic routing	Safety Recall: 71.7%	Rule-dependent logic; confined to medical domain
[4]	2025	Synthetic & Real Convos	Confidence-based intent routing	Sub-200ms target latency achieved	High dependency on intent embedding quality
[5]	2024	PopQA, TriviaQA	SELF-RAG: On-demand retrieval via reflection tokens	Outperforms Llama2 baselines in Factuality	Requires full LLM instruction fine-tuning
[6]	2024	PopQA, ARC	CRAG: Evaluator + Web Search fallback	Acc. improved by > 10%	Web search causes unpredictable latency spikes
[7]	2023	2WikiMultihop	FLARE: Forward-looking active retrieval	High Token Prob, EM on long-form	Needs LLM log-probs; generation paused
[8]	2024	Multi-domain QA	DRAGIN: Predictive asynchronous prefetching	End-to-end Latency reduced by 43.5%	Complex concurrent thread synchronization
[9]	2024	N/A (Theoretical)	Modular RAG taxonomy	Qualitative mapping of modular pipelines	Analytical framework only; no empirical algorithm
[10]	2024	Open-domain QA	RECOMP: Extractive & Abstractive compression	Compression Rate: 6% retained	Requires training a separate dedicated compressor
[11]	2024	LongBench	LongLLMLingua: Token-level perplexity pruning	Latency speedup: 1.4x–2.6x	Token pruning can disrupt syntax/grammar
[12]	2025	PubHealth, TriviaQA	Speculative RAG: Parallel subset drafting	Latency reduced by 50.83%	High concurrent compute resources needed
[13]	2025	Agent Benchmarks	ACON: Dynamic history context compression	Task Success Rate maintained w/ fewer tokens	Framework overhead unnecessary for simple queries
[14]	2023	GSM8K, BBH	LLMLingua: Prompt compression via budget LM	Up to 20x token compression	Small LM must compute full sequence perplexity

Ref.	Year	Dataset	Method / Architecture	Key Metric & Value	Stated Limitation
[15]	2024	Multi-hop QA	Auto-RAG: Autonomous iterative retrieval	Outperforms fixed k -iteration RAG	LLM judgment can cause infinite retrieval loops

2.1 Research Gaps

Based on the critical comparison of the literature above, the following research gaps emerge:

- **The All-or-Nothing Compression Penalty:** Advanced context compression models (e.g., LongLLMLingua [11]) effectively reduce token counts, but running the compression algorithm itself consumes compute. Current systems apply compression uniformly to all retrieved contexts, failing to bypass this computationally expensive step for simple queries that do not require it.
- **Latency Bottlenecks in LLM-based Routers:** State-of-the-art routing mechanisms (like SELF-RAG [5] and Auto-RAG [15]) rely on large language models to make routing decisions, establishing a high latency floor. There is a lack of research into using ultra-lightweight classifiers (e.g., DeBERTa) for initial sub-50ms triage.
- **Token Density vs. Latency Trade-offs at the Edge:** While token reduction is well-documented, precise analyses of the trade-off between the time taken to compress the context versus the time saved during the LLM generation phase in resource-constrained (edge) hardware remain scarce.

3 Problem Statement

Given a natural language user query and an unstructured text corpus (**Input**), the proposed system will aim to generate a contextually grounded natural language answer (**Output**), operating under the constraint of limited edge-compute hardware (e.g., single consumer GPU) which precludes massively parallel multi-agent loops or dynamic LLM generator fine-tuning (**Constraint**), such that the framework achieves a statistically significant reduction in average end-to-end latency and prompt token count on simple factual queries (PopQA) while maintaining or improving the Exact Match (EM) and F1 scores on complex multi-hop reasoning queries (HotpotQA) relative to a standard fixed-pipeline Naive RAG baseline (**Success Criterion**).

4 Proposed System Architecture

4.1 Block Diagram of Complete Pipeline

Data Acquisition (User Query) → Preprocessing (Query Normalization) → Model (Proposed Adaptive RAG Engine) → Post-Processing (Context Distillation) → Evaluation (Telemetry Logging) → Output (Answer Generation).

4.2 Detailed Model Diagram

Figure 1 illustrates the flow of the proposed architecture. The diagram is placed explicitly below to demonstrate the end-to-end pipeline, highlighting our novel contributions.

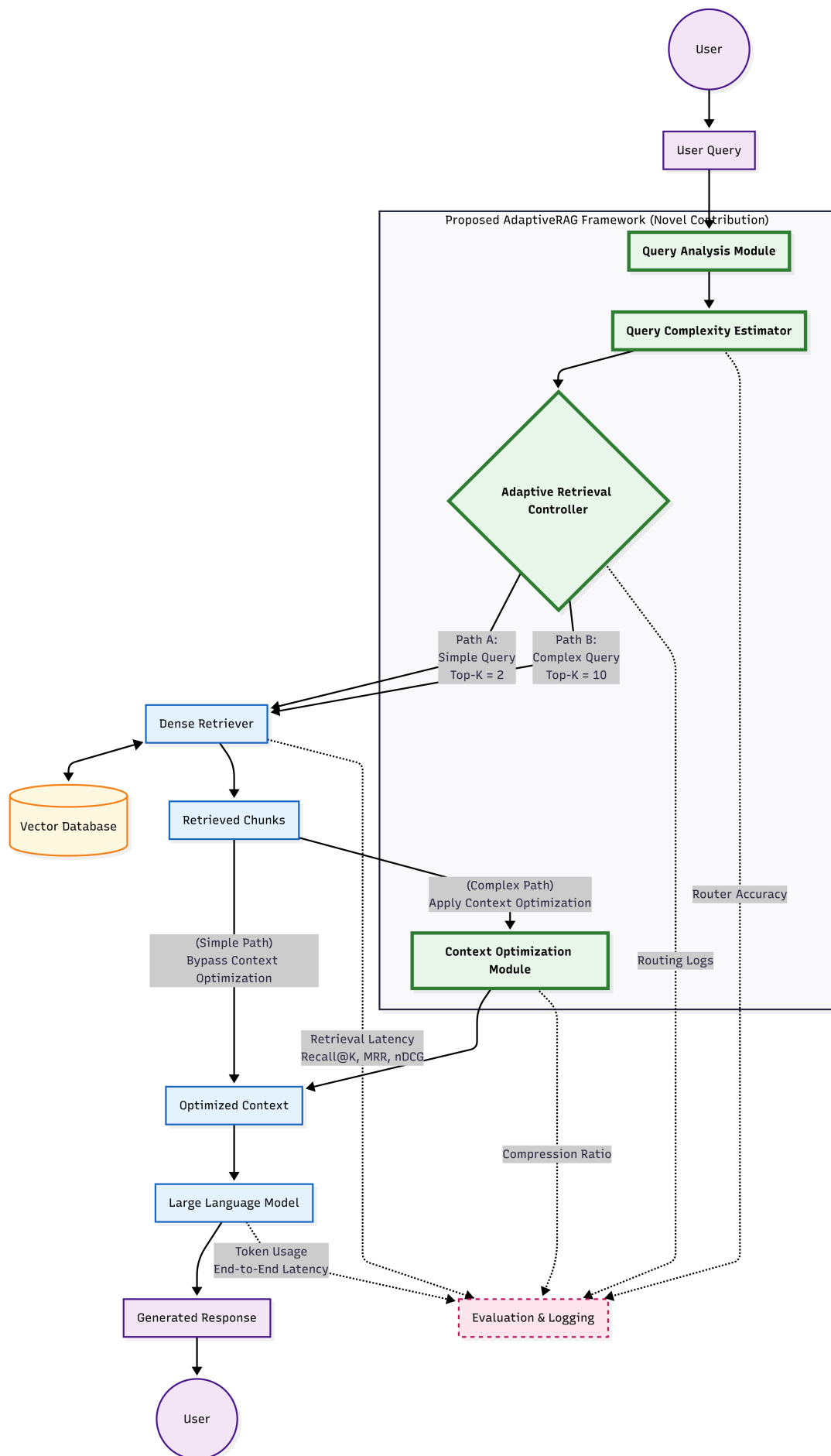


Figure 1: Proposed Adaptive RAG Framework featuring the conditional Context Optimization Module.

Detailed Diagram Description:

- **Query Analyzer & Complexity Estimator (Router):** The raw query will enter a lightweight sequence classifier (DeBERTa-v3-small). The output will be a binary route: Simple or Complex.
- **Adaptive Retrieval Controller:**
 - If *Simple*: Will set hyperparameter $k = 2$.
 - If *Complex*: Will set hyperparameter $k = 10$.
- **Vector Database (ChromaDB):** Will embed the query using `bge-small-en-v1.5` and fetch the top- k chunks.
- **Context Optimization Module (COM): NOVELTY PLACEMENT:** This represents the core proposed novelty. If the route is Simple, this module will be bypassed entirely. If Complex, the 10 chunks will be passed into the LLMingua compressor, which will calculate token-level perplexity and prune high-entropy (redundant) tokens.
- **Generator LLM:** The final synthesized context (either 2 raw chunks or 10 highly compressed chunks) will be fed alongside the query to the generator (`Phi-3-mini-4k-instruct`) to yield the final answer.

4.3 Justification for Design Choices

- **DeBERTa-v3 for the Router:** We propose using a small encoder model rather than prompting an LLM for routing because it processes text in $< 50\text{ms}$. This aims to address Research Gap 2 (LLM routing is too slow for latency-sensitive applications).
- **Conditional Bypassing of LLMingua:** Context compression takes time. We hypothesize that compressing a small context for a simple query will take longer than the generation time saved. Gating this module ensures compute is only spent when mathematically viable (aiming to address Research Gap 1).
- **Phi-3-mini as Generator:** Proposed for its exceptional reasoning-to-parameter ratio, which will allow it to easily fit into the VRAM constraints of the project while accurately following instructions based on the provided context.

4.4 Planned Experimental Setup

- **Datasets:** PopQA (for simple, factual queries) and HotpotQA (for complex, multi-hop queries).
- **Split Strategy:** A stratified subset of 1,000 queries from each dataset to allow rapid iteration and hardware-friendly benchmarking.
- **Evaluation Metrics:** Exact Match (EM) and F1-Score (for accuracy); End-to-end latency in milliseconds (for speed); Prompt token count (for efficiency).
- **Baselines:** (1) Naive RAG (fixed $k = 5$, no compression), and (2) Always-Compress RAG (fixed $k = 10$, always compresses).
- **Hardware:** Local execution on NVIDIA RTX 3060 / 4090 or Google Colab T4 GPU.
- **Ablation Studies:** We plan to disable the Context Optimization Module on the Complex route to measure the exact latency penalty and accuracy shift of passing $k = 10$ raw chunks directly to the generator.

5 Feasibility Note

Compute Available & Estimated Training Time:

The proposed architecture will rely entirely on frozen, pre-trained models (Phi-3, LLMingua, BGE embeddings). The only trainable component will be the DeBERTa complexity classifier. Training this classifier on a binary dataset of 2,000 samples is estimated to take less than 1 hour on a Google Colab T4 GPU. Inference for the entire pipeline is expected to fit within 8GB to 12GB of VRAM using 4-bit quantization.

Dataset Size & Access Status:

Both PopQA and HotpotQA are publicly available, open-source datasets accessible immediately via the HuggingFace datasets library. Data access poses zero risk.

Identified Risks & Fallback Plan:

- **Risk:** Out-Of-Memory (OOM) errors due to loading the Router, Embedding model, Compressor base model, and Generator LLM simultaneously on a single GPU.
- **Fallback Plan:** If local VRAM is exceeded, we will offload the Generator LLM (the heaviest component) to a free, high-speed API (e.g., Groq API for Llama-3-8B), while keeping the Router, Retriever, and Optimizer modules running locally to accurately measure pipeline routing logic.

References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems, 33, 9459-9474.
- [2] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). *Lost in the middle: How language models use long contexts*. Transactions of the Association for Computational Linguistics, 12, 157-173.
- [3] Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Generation to Query Complexity*. NAACL 2024.
- [4] Guo, Y., et al. (2025). *RouteRAG: Efficient Retrieval-Augmented Generation from Text and Graph via Reinforcement Learning*. arXiv preprint arXiv:2512.09487.
- [5] Lin, S., Shao, S., Liu, X., & Su, H. (2026). *FusionGraphRAG: An Adaptive Retrieval-Augmented Generation Framework for Complex Disease Management in the Elderly*. Information (MDPI), 17(2), 138.
- [6] Pattanayak, P., Agarwal, A., Meghwani, H., Patel, H. L., & Panda, S. (2025). *Hybrid AI for Responsive Multi-Turn Online Conversations with Novel Dynamic Routing and Feedback Adaptation*. NAACL Workshop 2025.
- [7] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). *SELF-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. ICLR 2024.
- [8] Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). *Corrective Retrieval Augmented Generation (CRAG)*. arXiv preprint arXiv:2401.15884.
- [9] Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., & Neubig, G. (2023). *Active Retrieval Augmented Generation (FLARE)*. EMNLP 2023.
- [10] Su, S., et al. (2024). *DRAGIN: Dynamic Retrieval Augmented Generation based on the Real-time Information Needs of Large Language Models*. arXiv preprint arXiv:2403.10081.
- [11] Gao, Y., Xiong, Y., Wang, M., & Wang, H. (2024). *Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks*. arXiv preprint arXiv:2407.21059.
- [12] Xu, F., Shi, W., & Choi, E. (2024). *RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation*. ICLR 2024.
- [13] Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., & Qiu, L. (2024). *LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression*. ACL 2024.
- [14] Wang, Z., Wang, Z., Le, L. T., Zheng, H. S., Mishra, S., Perot, V., Zhang, Y., Mattapalli, A., Taly, A., Shang, J., Lee, C.-Y., & Pfister, T. (2025). *Speculative RAG: Enhancing Retrieval Augmented Generation through Drafting*. ICLR 2025.
- [15] Kang, M., Chen, W.-N., Han, D., Inan, H. A., Wutschitz, L., Chen, Y., Sim, R., & Rajmohan, S. (2025). *ACON: Optimizing Context Compression for Long-horizon LLM Agents*. arXiv preprint arXiv:2510.00615.
- [16] Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., & Qiu, L. (2023). *LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models*. EMNLP 2023.
- [17] Yu, T., Zhang, S., & Feng, Y. (2024). *Auto-RAG: Autonomous Retrieval-Augmented Generation for Large Language Models*. arXiv preprint arXiv:2411.19443.